

02 — Lösungs-Design

Projekt: energie-planer · baut auf `01-discovery.md` auf

Lösungsidee

Ein **MCP-Server**, der einer KI (z.B. Claude) Werkzeuge gibt, um den manuellen Nachmittags-Job zu ersetzen. Der Schichtleiter fragt in normaler Sprache:

> „Wann soll ich morgen den Speicher 3 Stunden laden?“

Die KI ruft die Werkzeuge auf, bekommt **Live-Börsenpreise** + das **rechnerisch günstigste Fenster** zurück und erklärt den Plan. Aus 30 Minuten werden <2 Minuten.

Datenquelle

aWATTar-API — `https://api.awattar.at/v1/marketdata`

- öffentliche, **kostenlose** API, **ohne Key**, ausdrücklich für genau diese Nutzung gedacht → **kein Scraping, kein AGB-/Rechtsrisiko** (anders als z.B. AutoScout24).
- liefert stündliche Day-Ahead-Preise in **EUR/MWh** → wir rechnen in **ct/kWh** um (`EUR/MWh ÷ 10 = ct/kWh`).

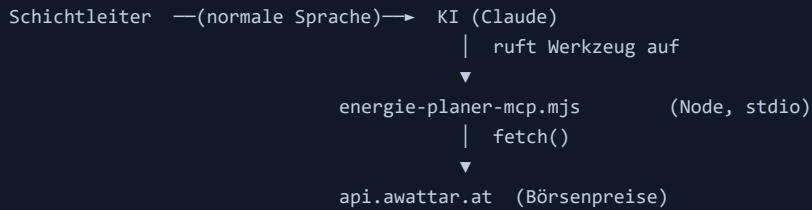
Tool-Schnitt (Prinzip: Werkzeug holt Daten + rechnet, Agent erklärt)

Leitregel: Ein Werkzeug tut nur, was die KI **nicht zuverlässig** kann — Live-Daten holen und **deterministisch** rechnen. Das Deuten/Empfehlen bleibt beim Agenten. Keine „Schein-Tools“, die nur LLM-Denken umhüllen.

Werkzeug	Eingabe	Ausgabe	Warum ein echtes Tool?	
<code>strompreise</code>	<code>{ tag: heute\</code>	<code>morgen }</code>	stündliche Preise (ct/kWh)	Live-Daten — kann die KI nicht
<code>guenstigstes_fenster</code>	<code>{ dauerStunden, fruehestens?, spaetestens?, tag? }</code>	bestes zusammenhängendes Fenster + Ersparnis vs. Tagesschnitt	deterministische Optimierung — die KI soll nicht „per Auge“ schätzen	
<code>negativpreis_stunden</code>	<code>{ tag? }</code>	Stunden mit negativem Preis	exakte Filterung über Live-Daten	

Direkter Bezug zur Discovery: `guenstigstes_fenster` nimmt `dauerStunden` (Speicher ~3 h) und `fruehestens` / `spaetestens` (Prozesse nur 06–22 Uhr) als Eingabe — genau die Randbedingungen aus `01-discovery.md`.

Architektur



- **Transport:** stdio (lokal). Kein Browser nötig → leichtgewichtig, später auch leicht

als HTTP-Server hostbar.

- **Laufzeit:** Node ≥ 18 (globales `fetch`). Abhängigkeiten: nur

`@modelcontextprotocol/sdk` + `zod`.

Bewusst NICHT im Scope (v1)

- Echtzeit-Speicherladestand / SCADA-Anbindung (käme in einer echten Folge-Iteration).
- Automatisches Schalten von Anlagen (v1 ist Entscheidungs-Unterstützung, kein Steuern).
- Mehrere Lasten gleichzeitig optimal verteilen (möglich als v2-Tool `lastplan`).

Produktions-Überlegungen (für das Bewerbungsgespräch)

- v1 läuft **lokal (stdio)** — richtig für Demo/Portfolio.
- In **Produktion** beim Kunden: als **HTTP-Server** auf einem Host (24/7), mit Token-Auth, Logging, und Anbindung an die reale Last-/Speicher-Steuerung.
- Rechtlich sauber, da öffentliche API mit erlaubter Nutzung.